REVISION CHEAT SHEET:
Transactions and Error Handling

Complete notes • examples • SQL • diagrams

INDEX — Main Topics Only

1. Transaction Fundamentals & Unit of Work
2. ACID Properties
3. Autocommit, Explicit & Implicit Transactions
4. BEGIN / COMMIT / ROLLBACK & Boundaries
5. @@TRANCOUNT & Nested Transactions
6. SAVE TRANSACTION & Partial Rollback
7. TRY...CATCH & Error Information Functions
8. THROW, RAISERROR & XACT_ABORT
9. XACT_STATE & Safe Transaction Patterns
10. Transaction Ownership in Nested Procedures
11. Isolation Levels & Row Versioning
12. Locks, Blocking & Deadlocks
13. Best Practices & Practical Fundraising Patterns
14. Interview Questions & Final Revision

Transactions and error handling belong together.
A transaction answers:

> Which database changes must succeed or fail as one unit?

Error handling answers:

> What should SQL Server do when something goes wrong during that unit of work?

For your FundraisingDb, the best example is creating a Gift.
A Gift may require:

EXAMPLE / FLOW

```
Create fundraising.Gift
      +
Create fundraising.GiftAllocation
```

Suppose a donor gives:

EXAMPLE / FLOW

$1,000

and the entire Gift goes to:

EXAMPLE / FLOW

Cancer Research Fund

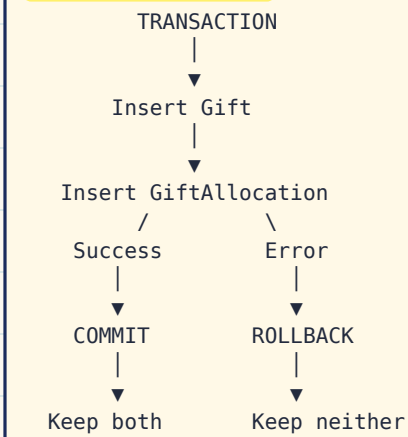
We do not want this:

EXAMPLE / FLOW

```
Gift inserted successfully
        ↓
GiftAllocation fails
        ↓
Database now contains an incomplete Gift
```

Instead we want:

EXAMPLE / FLOW



That is the central idea of transactions.



What Is a Transaction?

Simple Definition

A transaction is one logical unit of database work.
It may contain:

EXAMPLE / FLOW

one SQL statement

or:

EXAMPLE / FLOW

many SQL statements

that must be treated together.
If the transaction succeeds:

EXAMPLE / FLOW

COMMIT

makes the changes permanent.
If it fails:

EXAMPLE / FLOW

ROLLBACK

undoes the transaction's changes.
SQL Server supports autocommit, explicit, and implicit transaction modes.



Simple Transaction Example

Suppose:

EXAMPLE / FLOW

GiftId = 10000
TotalAmount = $1,000

FundId = 300
AllocationAmount = $1,000

We want:

SQL

```
BEGIN TRANSACTION;

INSERT INTO fundraising.Gift
(
    TenantId,
    ConstituentId,
    GiftDate,
    TotalAmount
)
VALUES
(
    1,
    1001,
    '2026-09-21',
    1000.00
);

-- Create related allocation here.
```

SQL • CONT.

```
COMMIT TRANSACTION;
```

Everything between:

EXAMPLE / FLOW

BEGIN TRANSACTION

and:

EXAMPLE / FLOW

COMMIT / ROLLBACK

belongs to the explicit transaction.



Why Are Transactions Needed?

Transactions protect related changes from becoming only partially completed.
Without a transaction:

EXAMPLE / FLOW

Statement 1 succeeds
Statement 2 succeeds
Statement 3 fails

You may be left with:

EXAMPLE / FLOW

partially changed data

With a correctly designed transaction:

EXAMPLE / FLOW

```
Statement 1
Statement 2
Statement 3
     │
     ├── all succeed → COMMIT
     │
     └── one fails   → ROLLBACK
```



Real Fundraising Example

Creating a Gift can involve:

EXAMPLE / FLOW

1. Insert Gift
2. Obtain GiftId
3. Insert GiftAllocation

These describe one business event:



A donor made a Gift and that money was allocated to a Fund.

Therefore they are natural candidates for one transaction.



When Should You Use a Transaction?

Use an explicit transaction when several operations must remain logically consistent together.

Examples:

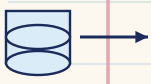
EXAMPLE / FLOW

Create Gift + GiftAllocation

Move money from one Fund balance
to another Fund balance

Create Constituent +
initial related records

Perform several related updates
that must succeed together



When Should You Not Create a Large Transaction?

Do not wrap unrelated work into one giant transaction.

Bad example:

EXAMPLE / FLOW

BEGIN TRANSACTION

Update Constituent

Generate report

Call external API

Wait for user response

Process 500,000 unrelated records

Send email

EXAMPLE / FLOW • CONT.

```
COMMIT
```

Long transactions can:

EXAMPLE / FLOW

hold locks longer

increase blocking

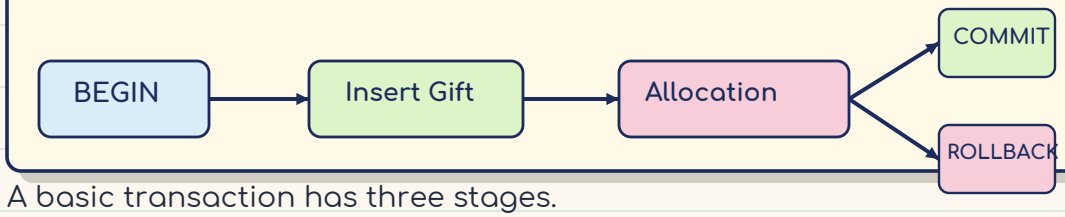
increase deadlock opportunity

consume transaction-log resources

delay other users

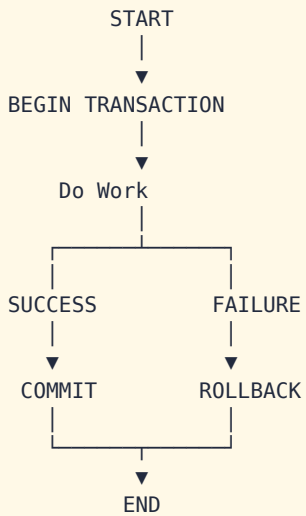
Keep the transaction as short as the business operation permits. Microsoft specifically recommends short transactions as a way to reduce blocking and deadlock risk.

Transaction Lifecycle



A basic transaction has three stages.

EXAMPLE / FLOW



EXAMPLE / FLOW • CONT.

Transaction Boundaries

A transaction boundary tells us where a transaction begins and ends.
Example:

SQL

```
BEGIN TRANSACTION;

UPDATE ...;
INSERT ...;
DELETE ...;

COMMIT TRANSACTION;
```

Boundary:

EXAMPLE / FLOW

```
BEGIN TRANSACTION
        ↓
     work
        ↓
COMMIT TRANSACTION
```

or:

EXAMPLE / FLOW

```
BEGIN TRANSACTION
        ↓
     work
        ↓
ROLLBACK TRANSACTION
```

ACID Properties

ACID Properties



Transactions are commonly explained using:

EXAMPLE / FLOW

A C I D

which means:

EXAMPLE / FLOW

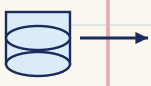
Atomicity

Consistency

Isolation

Durability

These are fundamental database transaction properties.



Atomicity

Simple Definition

Atomicity means all-or-nothing.
Either:

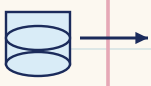
EXAMPLE / FLOW

all required operations succeed

or:

EXAMPLE / FLOW

none of them remain



Fundraising Example

Transaction:

EXAMPLE / FLOW

Insert Gift
+
Insert GiftAllocation

Scenario A:

EXAMPLE / FLOW

```
Gift succeeds
Allocation succeeds
        ↓
COMMIT
```

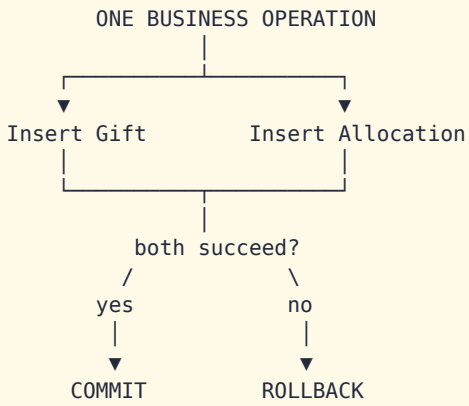
Scenario B:

EXAMPLE / FLOW

```
Gift succeeds
Allocation fails
        ↓
ROLLBACK
        ↓
Gift insert is undone too
```

Atomicity Diagram

EXAMPLE / FLOW



Why Atomicity Matters

Without atomicity, you could have:

EXAMPLE / FLOW

Gift exists

but

no allocation exists

even though the business requires allocations to accompany the Gift.

Consistency

Simple Definition

Consistency means a successful transaction takes the database from one valid state to another valid state.
Consistency is supported by things such as:

EXAMPLE / FLOW

PRIMARY KEY

FOREIGN KEY

CHECK constraints

UNIQUE constraints

NOT NULL rules

business logic

Fundraising Example

Suppose:

EXAMPLE / FLOW

GiftAllocation.FundId

must reference an existing:

EXAMPLE / FLOW

Fund.FundId

If somebody tries:

EXAMPLE / FLOW

FundId = 999999

and that Fund does not exist, the foreign key rejects the change.
The transaction should then handle that failure appropriately.

Consistency Does Not Mean

Consistency does not mean:

EXAMPLE / FLOW

every business rule is automatically
understood by SQL Server.

You must design:

EXAMPLE / FLOW

constraints

transactions

validation

procedures

correctly.



Isolation

Simple Definition

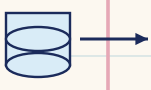
Isolation controls how concurrent transactions can see and affect each other's work.
Imagine two fundraisers using the system simultaneously.

EXAMPLE / FLOW

Transaction A
updates Gift 10000

Transaction B
tries to read/update Gift 10000

SQL Server must control this concurrency.
Isolation levels determine how reads interact with concurrent changes. SQL Server supports READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SNAPSHOT, and SERIALIZABLE; READ COMMITTED is the default isolation level for SQL Server.



Durability

Simple Definition

Durability means that after a successful durable COMMIT, the committed changes are expected to survive database/server failure and become permanent database state.
Example:

EXAMPLE / FLOW

```
Gift + Allocation
        ↓
COMMIT succeeds
        ↓
Transaction is complete
```

A normal fully durable transaction should not simply disappear

because SQL Server restarts.

ACID Memory Rule

EXAMPLE / FLOW

```
A = Atomicity
    All or nothing

C = Consistency
    Valid state → valid state

I = Isolation
    Control concurrent transactions

D = Durability
    Committed work persists
```

SQL Server Transaction Modes

The three modes you need to understand first are:

EXAMPLE / FLOW

Autocommit

Explicit Transaction

Implicit Transaction

SQL Server also has special batch-scoped behavior under MARS, but that is not the normal application transaction pattern.

Autocommit Transactions

Simple Definition

Autocommit means each individual SQL statement is its own transaction unless another transaction mode is active.
This is SQL Server's normal default behavior.
Example:

SQL

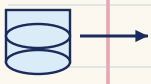
```
UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;
```

Conceptually SQL Server does:

EXAMPLE / FLOW

```
BEGIN
  UPDATE
COMMIT
```

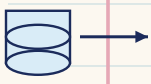
automatically if the statement succeeds.



Autocommit Flow

EXAMPLE / FLOW

```
UPDATE statement
      |
      ▼
SQL Server starts transaction
      |
      ▼
Execute UPDATE
    /      \
success    error
   |         |
   ▼         ▼
COMMIT    ROLLBACK
```



Autocommit Example with Two Statements

SQL

```
UPDATE fundraising.Gift
SET TotalAmount = 1000
WHERE GiftId = 10000;

UPDATE fundraising.GiftAllocation
SET AllocationAmount = 1000
WHERE GiftId = 10000;
```

Under normal autocommit behavior these are:

EXAMPLE / FLOW

```
Transaction 1
→ UPDATE Gift

Transaction 2
→ UPDATE GiftAllocation
```

They are not automatically one transaction.
If the first succeeds and second fails:

EXAMPLE / FLOW

first change may remain

That is why related multi-statement business operations often need an explicit transaction.



Explicit Transaction

Simple Definition

An explicit transaction is started manually using:

```sql
BEGIN TRANSACTION
```

and ended manually using:

```sql
COMMIT
```

or:

```sql
ROLLBACK
```



Explicit Transaction Example

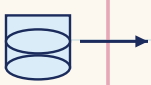
```sql
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1000
WHERE GiftId = 10000;

UPDATE fundraising.GiftAllocation
SET AllocationAmount = 1000
WHERE GiftId = 10000;

COMMIT TRANSACTION;
```

Now the two updates belong to one transaction.



Explicit Transaction Rule

Never write:

```sql
BEGIN TRANSACTION;

-- lots of code
```

without designing:

EXAMPLE / FLOW

Where does COMMIT happen?

Where does ROLLBACK happen?

What happens if an error occurs?



Implicit Transactions

Simple Definition

With implicit transaction mode enabled, SQL Server automatically starts a transaction when qualifying statements are executed, but you must explicitly COMMIT or ROLLBACK it.

Enable:

```sql
SET IMPLICIT_TRANSACTIONS ON;
```

Disable:

```sql
SET IMPLICIT_TRANSACTIONS OFF;
```

When implicit mode is on and @@TRANCOUNT = 0, statements such as SELECT against a table, INSERT, UPDATE, DELETE, CREATE, and ALTER TABLE can start a transaction automatically.



Implicit Example

```sql
SET IMPLICIT_TRANSACTIONS ON;

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;
```

You did not write:

```sql
BEGIN TRANSACTION;
```

but SQL Server starts one.

You still need:

```sql
COMMIT TRANSACTION;
```

or:

```sql
ROLLBACK TRANSACTION;
```

Danger of Implicit Transactions

A common mistake is:

EXAMPLE / FLOW

run UPDATE

forget COMMIT

walk away

The transaction can remain open and continue holding resources. For most application code, explicit transaction handling is usually easier to reason about.

Autocommit vs Explicit vs Implicit

Mode	Who starts it?	Who ends it?
Autocommit	SQL Server per statement	SQL Server
Explicit	Developer writes BEGIN	Developer writes COMMIT/ROLLBACK
Implicit	SQL Server after qualifying statement	Developer writes COMMIT/ROLLBACK

Memory:

EXAMPLE / FLOW

```
Autocommit
→ SQL Server handles start/end

Explicit
→ You handle start/end

Implicit
→ SQL Server starts,
  you must end
```

BEGIN TRANSACTION

Purpose

Starts an explicit transaction.

```sql
BEGIN TRANSACTION;
```

It increments:

EXAMPLE / FLOW

@@TRANCOUNT

by one.

Named Transaction

You can name a transaction.

```sql
BEGIN TRANSACTION CreateGift;
```

Example:

```sql
BEGIN TRANSACTION CreateGift;

INSERT INTO fundraising.Gift (...);

COMMIT TRANSACTION CreateGift;
```

Should You Name Every Transaction?

Not necessarily.
Names can improve readability in complex scripts, but they do not create independently managed nested transactions.
Also, the transaction name passed to COMMIT is largely documentary; SQL Server does not use it to independently commit a particular nested level.

COMMIT TRANSACTION

Simple Definition

COMMIT marks the successful completion of a transaction.

```sql
COMMIT TRANSACTION;
```

If @@TRANCOUNT = 1, the commit:

EXAMPLE / FLOW

makes the transaction's changes permanent

releases transaction resources

sets @@TRANCOUNT to 0

If @@TRANCOUNT > 1, an inner COMMIT only decreases @@TRANCOUNT; the outer transaction remains active.



COMMIT Example

```sql
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;

COMMIT TRANSACTION;
```

After the commit:

EXAMPLE / FLOW

you cannot simply ROLLBACK that transaction

because it is already committed.



ROLLBACK TRANSACTION

Simple Definition

ROLLBACK reverses changes made by the active transaction.

```sql
ROLLBACK TRANSACTION;
```

Example:

```sql
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 999999
WHERE GiftId = 10000;

ROLLBACK TRANSACTION;
```

The update is undone.



COMMIT vs ROLLBACK

EXAMPLE / FLOW

```
COMMIT
→ Keep transaction changes
```

EXAMPLE / FLOW

```
ROLLBACK
→ Undo transaction changes
```



Transaction Scope Across Multiple Statements

A transaction can cover many statements.

```sql
BEGIN TRANSACTION;

INSERT ...;

UPDATE ...;

DELETE ...;

INSERT ...;

COMMIT TRANSACTION;
```

All those statements participate in the same transaction on that connection.



Transaction Scope and Connections

A local transaction belongs to the connection/session that started it.
Transaction A on:

EXAMPLE / FLOW

Connection 1

is separate from work on:

EXAMPLE / FLOW

Connection 2

unless you deliberately involve distributed transaction mechanisms.
For normal .NET + SQL Server development, think:

EXAMPLE / FLOW

```
SqlConnection
      |
      ▼
Transaction
      |
      ▼
Commands on that connection
```



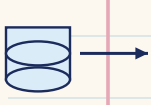
@@TRANCOUNT

Simple Definition

@@TRANCOUNT tells you how many BEGIN TRANSACTION levels currently exist on the connection.
Example:

SQL

```
SELECT @@TRANCOUNT;
```



@@TRANCOUNT Example

SQL

```
SELECT @@TRANCOUNT;
-- 0

BEGIN TRANSACTION;

SELECT @@TRANCOUNT;
-- 1

BEGIN TRANSACTION;

SELECT @@TRANCOUNT;
-- 2

COMMIT TRANSACTION;
```

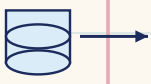
SQL • CONT.

```
SELECT @@TRANCOUNT;
-- 1

COMMIT TRANSACTION;

SELECT @@TRANCOUNT;
-- 0
```

BEGIN TRANSACTION increments the count, while COMMIT decrements it. A normal full ROLLBACK resets it to zero; rolling back to a savepoint does not change @@TRANCOUNT.



Nested Transactions

Simple Definition

SQL Server allows multiple:

SQL

```
BEGIN TRANSACTION
```

statements.
This can look like nested transactions.
Example:

SQL

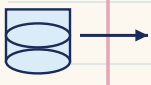
```
BEGIN TRANSACTION OuterTran;

    BEGIN TRANSACTION InnerTran;

    COMMIT TRANSACTION InnerTran;

COMMIT TRANSACTION OuterTran;
```

But there is a critical rule.



SQL Server Does Not Have Independently Committable Nested Transactions

This is extremely important.
SQL Server does not give you two completely independent transactions.
Example:

EXAMPLE / FLOW

```
BEGIN Outer
  @@TRANCOUNT = 1

BEGIN Inner
  @@TRANCOUNT = 2
```

EXAMPLE / FLOW • CONT.

```
COMMIT Inner
  @@TRANCOUNT = 1
```

The inner commit does not make that inner work permanently committed.
Only the outermost commit completes the transaction.



Nested Transaction Diagram

EXAMPLE / FLOW

```
BEGIN Outer
│
│   @@TRANCOUNT = 1
│
├── BEGIN Inner
│       │
│       │ @@TRANCOUNT = 2
│       │
│       └── COMMIT Inner
│               │
│               │ @@TRANCOUNT = 1
│               │
│               └── NOT permanently committed yet
│
└── COMMIT Outer
        │
        ▼
   Real commit
```



Nested ROLLBACK Surprise

Consider:

SQL

```
BEGIN TRANSACTION;

    BEGIN TRANSACTION;

    ROLLBACK TRANSACTION;
```

A normal rollback does not simply roll back the "inner transaction".
It rolls back the entire active transaction and sets:

EXAMPLE / FLOW

```
@@TRANCOUNT = 0
```

unless you explicitly roll back to a savepoint.

Nested Transaction Memory Rule

EXAMPLE / FLOW

```
Nested BEGIN
→ Increases @@TRANCOUNT

Inner COMMIT
→ Decreases count only

Full ROLLBACK
→ Rolls everything back

Outermost COMMIT
→ Actually completes transaction
```

SAVE TRANSACTION

Simple Definition

A savepoint marks a point inside an existing transaction.
Syntax:

SQL

```
SAVE TRANSACTION SavePointName;
```

Later:

SQL

```
ROLLBACK TRANSACTION SavePointName;
```

can roll back work after that savepoint without necessarily rolling back the whole transaction.

Fundraising Savepoint Example

Suppose:

EXAMPLE / FLOW

Step 1
Update Gift

Step 2
Try optional allocation adjustment

Step 3
Continue transaction

Example:

SQL

```
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1200
WHERE GiftId = 10000;

SAVE TRANSACTION BeforeAllocationChange;

UPDATE fundraising.GiftAllocation
SET AllocationAmount = 1200
WHERE GiftId = 10000;

-- If only the allocation change should be undone:
ROLLBACK TRANSACTION BeforeAllocationChange;

COMMIT TRANSACTION;
```



Partial Rollback Flow

EXAMPLE / FLOW

```
BEGIN TRANSACTION
      |
      ▼
Update Gift
      |
      ▼
SAVE BeforeAllocation
      |
      ▼
Update Allocation
      |
   error/decision
      |
      ▼
ROLLBACK TO SAVEPOINT
      |
      ▼
Gift update remains
      |
      ▼
COMMIT
```



Important Savepoint Rule

A rollback to a savepoint:

EXAMPLE / FLOW

does not reset @@TRANCOUNT

because the transaction itself is still active.

When to Use Savepoints

Use savepoints when:

EXAMPLE / FLOW

one larger transaction must remain active

but a later optional portion may need
partial rollback

Do not use them merely because the feature exists.
For most straightforward business operations:

EXAMPLE / FLOW

all succeed
or
all roll back

is simpler.

Transaction Naming vs Savepoint Naming

Do not confuse:

EXAMPLE / FLOW

Transaction name

with:

EXAMPLE / FLOW

Savepoint name

Transaction:

SQL

```
BEGIN TRANSACTION CreateGift;
```

Savepoint:

SQL

```
SAVE TRANSACTION BeforeAllocation;
```

A savepoint is the real mechanism used for partial rollback.

Error Handling Fundamentals

A transaction tells SQL Server:

> These changes belong together.

Error handling tells SQL Server:

> What should we do if execution fails?

The main SQL Server error-handling tools are:

EXAMPLE / FLOW

TRY...CATCH

THROW

RAISERROR

ERROR_MESSAGE()

ERROR_NUMBER()

ERROR_SEVERITY()

ERROR_STATE()

ERROR_LINE()

ERROR_PROCEDURE()

TRY...CATCH

Simple Definition

TRY...CATCH lets you attempt SQL statements and transfer control to an error-handling block when a catchable run-time error occurs.

Syntax:

```sql
BEGIN TRY

    -- statements that may fail

END TRY
BEGIN CATCH

    -- error handling

END CATCH;
```

Simple Example

SQL

```
BEGIN TRY

    SELECT 1 / 0;

END TRY
BEGIN CATCH

    SELECT
        ERROR_MESSAGE() AS ErrorMessage;

END CATCH;
```

The divide-by-zero error causes control to enter:

EXAMPLE / FLOW

CATCH

TRY...CATCH Flow

EXAMPLE / FLOW

```
BEGIN TRY
    |
    ▼
Run statement
   /    \
okay    error
 |        |
 ▼        ▼
next     CATCH
statement |
          ▼
      handle error
```

Error Information Functions

Inside CATCH, SQL Server gives you:

EXAMPLE / FLOW

ERROR_NUMBER()

ERROR_MESSAGE()

ERROR_SEVERITY()

ERROR_STATE()

ERROR_LINE()

EXAMPLE / FLOW • CONT.

ERROR_PROCEDURE()

Microsoft documents these as the standard functions for retrieving caught-error information.

ERROR_NUMBER()

Returns the SQL Server error number.

```sql
ERROR_NUMBER()
```

Examples could include:

EXAMPLE / FLOW

constraint violation number

conversion error number

divide-by-zero error number

ERROR_MESSAGE()

Returns the text describing the error.

```sql
ERROR_MESSAGE()
```

Example conceptually:

EXAMPLE / FLOW

The INSERT statement conflicted
with the FOREIGN KEY constraint...

ERROR_SEVERITY()

Returns the error's severity.
Severity describes the general seriousness/category of the error.
For normal application programming:



You usually inspect or log the severity rather than writing complicated application logic around every severity number.

ERROR_STATE()

Returns the error's state value.
State can help distinguish different places or conditions that raise the same error number.
Custom THROW also accepts a state value.

ERROR_LINE()

Returns the line number where the error occurred.
Very useful for debugging procedures and larger scripts.

ERROR_PROCEDURE()

Returns the stored procedure or trigger in which the error occurred when that information is available.
Useful for logging:

EXAMPLE / FLOW

Which procedure failed?

Complete Error Information Example

SQL

```
BEGIN TRY

    SELECT 1 / 0;

END TRY
BEGIN CATCH

    SELECT
        ERROR_NUMBER()    AS ErrorNumber,
        ERROR_MESSAGE()   AS ErrorMessage,
        ERROR_SEVERITY()  AS ErrorSeverity,
        ERROR_STATE()     AS ErrorState,
        ERROR_LINE()      AS ErrorLine,
        ERROR_PROCEDURE() AS ErrorProcedure;

END CATCH;
```

THROW

Simple Definition

THROW raises an exception.
Two important forms exist.
Create your own error:

```sql
THROW 50001,
      'Gift amount must be positive.',
      1;
```

Rethrow the current error from inside CATCH:

```sql
THROW;
```

Parameterless THROW

Inside:

```sql
BEGIN CATCH
```

use:

```sql
THROW;
```

to rethrow the original error.
This preserves the original error information much better than replacing it with a generic message.

Validation Example with THROW

```sql
IF @TotalAmount <= 0
BEGIN
    THROW 50001,
          'TotalAmount must be greater than zero.',
          1;
END;
```

THROW Flow

EXAMPLE / FLOW

```
Invalid business input
        |
        ▼
THROW
        |
        ▼
Execution stops
        |
```

EXAMPLE / FLOW • CONT.

```
        ▼
Nearest CATCH
or calling application
receives error
```

RAISERROR

RAISERROR is the older SQL Server error-raising mechanism.
Example:

SQL

```
RAISERROR
(
    'Gift amount is invalid.',
    16,
    1
);
```

It supports useful legacy features such as:

EXAMPLE / FLOW

custom severity

custom state

formatted messages

NOWAIT

LOG

But for new application code, Microsoft recommends THROW rather than RAISERROR; importantly, THROW honors SET XACT_ABORT, while RAISERROR does not.

THROW vs RAISERROR

THROW	RAISERROR
Modern recommended choice	Older mechanism
Can rethrow original error	Common in legacy code
Honors XACT_ABORT	Does not honor XACT_ABORT
Simpler error syntax	More formatting/severity options
Best default for new application code	Know for maintenance/interviews

Memory:

EXAMPLE / FLOW

```
New code
→ THROW

Legacy SQL
→ You may still encounter RAISERROR
```



SET XACT_ABORT

This is one of the most important transaction settings.
Syntax:

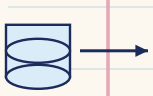
SQL

```
SET XACT_ABORT ON;
```

or:

SQL

```
SET XACT_ABORT OFF;
```



XACT_ABORT ON

Simple Definition

When XACT_ABORT is ON, many run-time errors cause SQL Server to terminate and roll back the transaction rather than merely failing one statement.
Microsoft describes ON as terminating and rolling back the transaction for run-time errors covered by the setting.



XACT_ABORT OFF

OFF is the default for ordinary T-SQL batches.
With OFF, some errors roll back only the failing statement while leaving the transaction active.
However:



Do not interpret OFF as "errors never roll back transactions."
Different errors have different effects.



XACT_ABORT Example Concept

EXAMPLE / FLOW

BEGIN TRANSACTION

Statement 1 succeeds

Statement 2 fails

Statement 3

With:

EXAMPLE / FLOW

XACT_ABORT OFF

some error types may allow:

EXAMPLE / FLOW

Statement 1 remains pending
Statement 2 is rejected
Statement 3 continues

With:

EXAMPLE / FLOW

XACT_ABORT ON

a covered run-time error can cause:

EXAMPLE / FLOW

```
whole transaction → rollback
```



Why Use XACT_ABORT ON?

For many stored procedures that perform an atomic transactional operation, this is a strong pattern:

SQL

```
SET XACT_ABORT ON;
```

because it reduces the chance of accidentally continuing after certain run-time errors.
But you still need:

EXAMPLE / FLOW

TRY...CATCH

ROLLBACK logic

THROW

for controlled error handling and caller notification.

THROW and XACT_ABORT

Important:

EXAMPLE / FLOW

```
THROW
→ honors SET XACT_ABORT
```

while:

EXAMPLE / FLOW

```
RAISERROR
→ does not
```

XACT_STATE()

Simple Definition

XACT_STATE() tells you whether the current session has:

EXAMPLE / FLOW

No transaction

A committable transaction

An uncommittable transaction

Possible results:

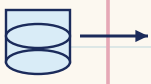
EXAMPLE / FLOW

1
0
-1

XACT_STATE = 0

EXAMPLE / FLOW

No active user transaction



XACT_STATE = 1

EXAMPLE / FLOW

An active transaction exists

AND

it is committable

This means SQL Server still considers it valid for:

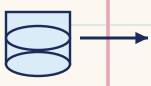
EXAMPLE / FLOW

COMMIT

or:

EXAMPLE / FLOW

ROLLBACK



XACT_STATE = -1

EXAMPLE / FLOW

An active transaction exists

BUT

it is uncommittable

This is sometimes informally called a:

EXAMPLE / FLOW

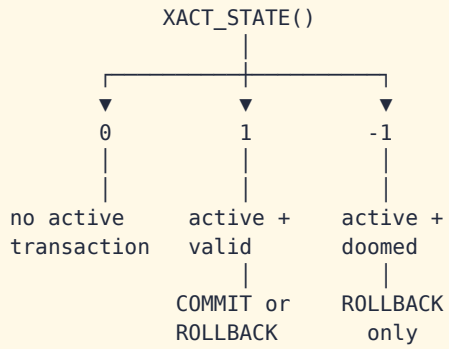
doomed transaction

In this state you cannot successfully commit it or perform normal writes.
It must be rolled back.



XACT_STATE Diagram

EXAMPLE / FLOW



XACT_STATE vs @@TRANCOUNT

These answer different questions.
@@TRANCOUNT asks:

> How many transaction levels have been started?

XACT_STATE() asks:

> Is the current transaction usable/committable?

Microsoft explicitly notes that @@TRANCOUNT cannot tell whether an active transaction is uncommittable, while XACT_STATE() does not tell the nesting count.

Comparison

Function	Main Question
@@TRANCOUNT	Is a transaction open, and what is the nesting count?
XACT_STATE()	Is the active transaction committable?

Committable vs Uncommittable Transaction

Committable

EXAMPLE / FLOW

XACT_STATE() = 1

You may:

EXAMPLE / FLOW

COMMIT

or

ROLLBACK

Uncommittable

EXAMPLE / FLOW

XACT_STATE() = -1

You cannot:

EXAMPLE / FLOW

COMMIT

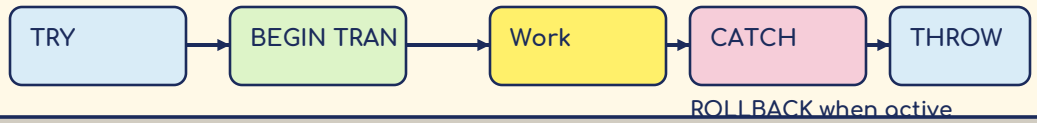
and should not attempt further normal writes.
You need:

EXAMPLE / FLOW

ROLLBACK

Basic TRY...CATCH Transaction Pattern

Basic TRY...CATCH Transaction Pattern



SQL

```
SET XACT_ABORT ON;

BEGIN TRY

    BEGIN TRANSACTION;

    -- database changes

    COMMIT TRANSACTION;

END TRY
BEGIN CATCH
```

SQL • CONT.

```
    IF XACT_STATE() <> 0
    BEGIN
        ROLLBACK TRANSACTION;
    END;

    THROW;

END CATCH;
```

This is one of the most important patterns in this chapter.

Understand the Pattern Step by Step

EXAMPLE / FLOW

```
SET XACT_ABORT ON
        |
        ▼
BEGIN TRY
        |
        ▼
BEGIN TRANSACTION
        |
        ▼
Database work
      /     \
 success    error
   |          |
   ▼          ▼
COMMIT       CATCH
               |
               ▼
      transaction active?
               |
               ▼
          ROLLBACK
               |
               ▼
             THROW
```

Complete Fundraising Example

Requirement:

Create a Gift and one GiftAllocation safely.

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_CreateGiftWithAllocation
    @TenantId BIGINT,
    @ConstituentId BIGINT,
    @CampaignId BIGINT = NULL,
    @FundId BIGINT,
    @GiftDate DATE,
```

SQL • CONT.

```
    @TotalAmount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    IF @TotalAmount <= 0
    BEGIN
        THROW 50001,
              'TotalAmount must be greater than zero.',
              1;
    END;

    BEGIN TRY

        BEGIN TRANSACTION;

        INSERT INTO fundraising.Gift
        (
            TenantId,
            ConstituentId,
            CampaignId,
            GiftDate,
            TotalAmount
        )
        VALUES
        (
            @TenantId,
            @ConstituentId,
            @CampaignId,
            @GiftDate,
            @TotalAmount
        );

        DECLARE @GiftId BIGINT =
            CONVERT
            (
                BIGINT,
                SCOPE_IDENTITY()
            );

        INSERT INTO fundraising.GiftAllocation
        (
            TenantId,
            GiftId,
            FundId,
            AllocationAmount
        )
        VALUES
        (
            @TenantId,
            @GiftId,
            @FundId,
            @TotalAmount
        );

        COMMIT TRANSACTION;

        SELECT
            @GiftId AS GiftId;

    END TRY
    BEGIN CATCH
```

SQL • CONT.

```
        IF XACT_STATE() <> 0
        BEGIN
            ROLLBACK TRANSACTION;
        END;

        THROW;

    END CATCH;
END;
GO
```



Why This Pattern Is Safe

If:

EXAMPLE / FLOW

Gift INSERT succeeds

but:

EXAMPLE / FLOW

GiftAllocation INSERT fails

control goes to:

EXAMPLE / FLOW

CATCH

then:

EXAMPLE / FLOW

ROLLBACK

undoes the Gift INSERT too.
The caller receives the error because:

SQL

```
THROW;
```

rethrows it.



Why Not Silently Catch the Error?

Bad:

```sql
BEGIN CATCH
    PRINT 'Error';
END CATCH;
```

The application might think:

EXAMPLE / FLOW

operation succeeded

when it did not.

Better:

EXAMPLE / FLOW

rollback
+
throw



Transaction-Safe Stored Procedures

This needs special care when one stored procedure may be called:

EXAMPLE / FLOW

directly

or:

EXAMPLE / FLOW

from another procedure that already
has a transaction



The Problem

Suppose:

EXAMPLE / FLOW

```
Procedure A
    |
    ├── BEGIN TRANSACTION
    |
    └── calls Procedure B
```

If Procedure B blindly does:

EXAMPLE / FLOW

BEGIN TRANSACTION

and later:

EXAMPLE / FLOW

ROLLBACK

it may roll back Procedure A's entire transaction.
Remember:

SQL Server does not provide independently manageable nested transactions.

Beginner-Safe Rule

If the procedure always owns the full business transaction:

EXAMPLE / FLOW

BEGIN
COMMIT
ROLLBACK

inside that procedure is straightforward.
If the procedure may participate inside a caller's transaction:

You need to know who owns the transaction.

Advanced Transaction-Safe Procedure Pattern

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_DoTransactionalWork
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @StartedTransaction BIT = 0;

    IF @@TRANCOUNT = 0
    BEGIN
        BEGIN TRANSACTION;
        SET @StartedTransaction = 1;
    END
    ELSE
    BEGIN
        SAVE TRANSACTION ProcedureSavePoint;
    END;
```

SQL • CONT.

```
    BEGIN TRY

        -- Do database work here.

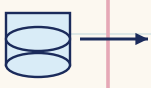
        IF @StartedTransaction = 1
        BEGIN
            COMMIT TRANSACTION;
        END;

    END TRY
    BEGIN CATCH

        IF XACT_STATE() = -1
        BEGIN
            -- Transaction cannot be saved.
            -- If this procedure owns it, roll it back.
            IF @StartedTransaction = 1
            BEGIN
                ROLLBACK TRANSACTION;
            END;
        END
        ELSE IF XACT_STATE() = 1
        BEGIN
            IF @StartedTransaction = 1
            BEGIN
                ROLLBACK TRANSACTION;
            END
            ELSE
            BEGIN
                ROLLBACK TRANSACTION ProcedureSavePoint;
            END;
        END;

        THROW;

    END CATCH;
END;
GO
```



Understand This Pattern

First ask:

EXAMPLE / FLOW

Is there already a transaction?

If:

EXAMPLE / FLOW

@@TRANCOUNT = 0

the procedure starts one and becomes its owner.

If:

EXAMPLE / FLOW

@@TRANCOUNT > 0

the caller already owns a transaction.
The procedure creates a:

EXAMPLE / FLOW

SAVEPOINT

instead.

Flow

EXAMPLE / FLOW

```
Procedure starts
      |
      ▼
@@TRANCOUNT = 0 ?
   /             \
 YES              NO
  |                |
  ▼                ▼
BEGIN TRAN      SAVEPOINT
own transaction join caller's transaction
  |                |
  └──────┬─────────┘
         ▼
        Work
       /    \
   success  error
      |       |
      ▼       ▼
if owner:   CATCH
COMMIT       |
             ▼
   decide full rollback
   or savepoint rollback
```

This pattern is advanced. Learn the basic transaction pattern first, then use this when procedure nesting genuinely requires it.

Isolation Levels

Isolation is part of ACID and directly affects locking and concurrency.
SQL Server isolation levels:

EXAMPLE / FLOW

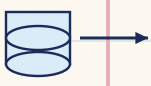
READ UNCOMMITTED

READ COMMITTED

REPEATABLE READ

SNAPSHOT

SERIALIZABLE



READ UNCOMMITTED

Allows:

EXAMPLE / FLOW

dirty reads

A transaction can read another transaction's uncommitted data.
Example:

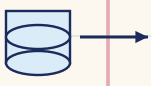
EXAMPLE / FLOW

Transaction A:
changes Gift from $1,000 to $50,000
but has not committed.

Transaction B under READ UNCOMMITTED:
may read $50,000.

Transaction A then rolls back.

Transaction B read a value that never became permanent.



Should You Use READ UNCOMMITTED / NOLOCK Everywhere?

No.
It may reduce some blocking but allows inconsistent/uncommitted reads.
For financial/fundraising data:

EXAMPLE / FLOW

incorrect donor totals

may be much worse than waiting briefly for a lock.

READ COMMITTED

This is the normal default isolation level in SQL Server.
It prevents:

EXAMPLE / FLOW

dirty reads

but another transaction may change data between two separate reads.
Depending on the database's READ_COMMITTED_SNAPSHOT setting, READ COMMITTED can use locking or row versioning. On SQL Server, READ_COMMITTED_SNAPSHOT is off by default; when enabled, reads use row versions rather than traditional shared-lock behavior.

REPEATABLE READ

Prevents another transaction from modifying rows that your transaction has read until your transaction completes.
It prevents:

EXAMPLE / FLOW

dirty reads

nonrepeatable reads

but new matching rows may still appear:

EXAMPLE / FLOW

phantom rows

SERIALIZABLE

Most restrictive locking-based level.
It also protects key ranges so another transaction cannot insert rows into ranges your transaction has read.
It prevents:

EXAMPLE / FLOW

dirty reads

nonrepeatable reads

phantom reads

but may reduce concurrency significantly.

SNAPSHOT

Uses row versioning.
A transaction reads a transactionally consistent version of data from the start of the transaction rather than being blocked by many concurrent writers.
It requires:

EXAMPLE / FLOW

```
ALLOW_SNAPSHOT_ISOLATION ON
```

for the database before using SNAPSHOT.

Isolation Comparison

Level	Dirty Read	Nonrepeatable Read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SNAPSHOT	Prevented	Prevented	Prevented
SERIALIZABLE	Prevented	Prevented	Prevented

Do Not Automatically Choose the Highest Isolation

Higher isolation can provide stronger consistency guarantees, but usually increases:

EXAMPLE / FLOW

locking

blocking

resource use

concurrency conflicts

Choose the level according to the actual business consistency requirement.

Locking

Simple Definition

A lock is one mechanism SQL Server uses to coordinate concurrent

access to data.
For example:

EXAMPLE / FLOW

Transaction A
updates Gift 10000

SQL Server protects that change so another transaction cannot perform conflicting work simultaneously.
Locking helps SQL Server preserve transactional consistency.

Locking Flow

EXAMPLE / FLOW

```
Transaction A
UPDATE Gift 10000
      |
      ▼
SQL Server obtains required lock
      |
      ▼
Transaction B wants conflicting lock
      |
      ▼
B waits
      |
      ▼
A commits/rolls back
      |
      ▼
Lock released
      |
      ▼
B continues
```

Lock Granularity

SQL Server can work with locks at different resource levels, for example:

EXAMPLE / FLOW

Row/key

Page

Table

Database/schema-related resources

The Database Engine normally chooses locking behavior automatically.
Do not manually force lock hints everywhere unless there is a specific,

measured reason.

Lock Duration

Lock duration depends on:

EXAMPLE / FLOW

type of operation

transaction isolation level

locking/versioning configuration

whether optimized locking is enabled

Write protection is particularly important because transactions need to protect modified data until transaction completion; SQL Server 2025's optimized locking can reduce the duration/number of row and page locks in supported configurations, but transaction design still matters.

Blocking

Simple Definition

Blocking occurs when one session must wait because another transaction holds an incompatible lock.

Blocking is not automatically an error.

Example:

EXAMPLE / FLOW

Transaction A
updates Gift 10000

Transaction B
tries to update Gift 10000

B waits for A

This is normal concurrency control.

When Blocking Becomes a Problem

Blocking becomes problematic when:

EXAMPLE / FLOW

transactions remain open too long

queries touch too many rows

indexes are poor

EXAMPLE / FLOW • CONT.

applications pause while holding transactions

transactions are abandoned

Deadlocks

Simple Definition

A deadlock occurs when two or more transactions are waiting on each other in a cycle and none can continue.
Example:

EXAMPLE / FLOW

Transaction A
locks Gift 10000
then wants Fund 300

Transaction B
locks Fund 300
then wants Gift 10000

Diagram:

EXAMPLE / FLOW

```
Transaction A
   | owns Gift
   ▼
Gift 10000
   ▲
   | wants
Transaction B

Transaction B
   | owns Fund
   ▼
Fund 300
   ▲
   | wants
Transaction A
```

Neither can move.
SQL Server detects the deadlock and chooses a victim to roll back so the other transaction can continue.

Reducing Deadlocks

Good practices:

EXAMPLE / FLOW

Keep transactions short

Access objects in consistent order

Use appropriate indexes

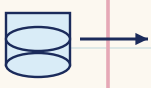
Avoid unnecessary high isolation levels

Avoid user interaction inside transaction

Avoid unnecessary network round-trips
inside transactions

Retry appropriate deadlock-victim failures
at the application layer

Microsoft specifically recommends short transactions, consistent access patterns, and avoiding unnecessarily high isolation levels when reducing deadlocks.



Consistent Access Order Example

Bad:

EXAMPLE / FLOW

```
Transaction A:
Gift → Fund

Transaction B:
Fund → Gift
```

Better:

EXAMPLE / FLOW

```
Transaction A:
Gift → Fund

Transaction B:
Gift → Fund
```

Consistent resource ordering can reduce circular wait patterns.



Transaction Duration

A transaction begins at its transaction boundary and remains open until:

EXAMPLE / FLOW

COMMIT

or:

EXAMPLE / FLOW

ROLLBACK

The longer it stays open, the longer transaction-related resources may be retained.



Bad Long Transaction

EXAMPLE / FLOW

BEGIN TRANSACTION

Update Gift

wait 30 seconds

call API

wait for response

update allocation

COMMIT

Avoid this.



Better Pattern

Do external work first when business semantics permit:

EXAMPLE / FLOW

Validate application request

Call required service

Prepare values

BEGIN TRANSACTION

Perform database changes

COMMIT

The transaction contains only the short atomic database section.

Open Transaction Mistake

One of the worst mistakes:

SQL

```
BEGIN TRANSACTION;

UPDATE fundraising.Gift
SET TotalAmount = 1000
WHERE GiftId = 10000;

-- forgot COMMIT / ROLLBACK
```

Now:

EXAMPLE / FLOW

@@TRANCOUNT > 0

and the transaction stays open on that connection.
This can cause:

EXAMPLE / FLOW

blocking

locks/resources remaining active

transaction-log growth

unexpected later behavior

Detecting an Open Transaction

SQL

```
SELECT
    @@TRANCOUNT AS TransactionCount,
    XACT_STATE() AS TransactionState;
```

Example:

EXAMPLE / FLOW

@@TRANCOUNT = 1
XACT_STATE() = 1

means:

EXAMPLE / FLOW

active committable transaction

Never Assume CATCH Automatically Rolls Back

This is very important.
TRY...CATCH does not mean:

EXAMPLE / FLOW

SQL Server always rolled back everything for you.

Inside CATCH, inspect the transaction state and perform the required rollback.

Common Safe CATCH Pattern

```sql
BEGIN CATCH

    IF XACT_STATE() <> 0
    BEGIN
        ROLLBACK TRANSACTION;
    END;

    THROW;

END CATCH;
```

Why IF @@TRANCOUNT > 0 ROLLBACK Is Also Common

You may see:

```sql
IF @@TRANCOUNT > 0
BEGIN
    ROLLBACK TRANSACTION;
END;
```

This detects whether a transaction is active.
For more detailed logic involving committable vs uncommittable states:

EXAMPLE / FLOW

XACT_STATE()

provides more information.

Error Handling Outside Transactions

Not every TRY...CATCH requires a transaction.

Example:

```sql
BEGIN TRY

    SELECT
        CAST('ABC' AS INT);

END TRY
BEGIN CATCH

    SELECT
        ERROR_MESSAGE();

END CATCH;
```

Transactions are needed when database changes need transactional behavior, not simply because an error might occur.



Compile Errors vs Runtime Errors

TRY...CATCH is mainly for catchable execution/runtime errors.
Some errors occurring at the same execution level, including certain compilation/name-resolution errors, may not be caught by the surrounding TRY...CATCH in the way beginners expect.
Also, SET XACT_ABORT does not affect compile-time errors such as syntax errors.



Error Handling Does Not Replace Constraints

Do not write only:

EXAMPLE / FLOW

IF Fund exists

and remove the foreign key.
Use:

EXAMPLE / FLOW

Foreign key
+
appropriate validation
+
TRY...CATCH

The constraint is the final data-integrity protection.

Transaction Does Not Replace Validation

A transaction makes work atomic.
It does not tell SQL Server:

EXAMPLE / FLOW

TotalAmount must be positive

You still need:

EXAMPLE / FLOW

CHECK constraint

procedure validation

application validation

as appropriate.

Important Transaction Mistakes

Mistake 1 — Forgetting COMMIT or ROLLBACK

Always close every transaction path.

Mistake 2 — Assuming Inner COMMIT Is Final

It is not.
Only the outermost commit completes the transaction.

Mistake 3 — Full ROLLBACK Inside a Nested Procedure

A plain rollback can roll back the caller's transaction too.
Understand transaction ownership.

Mistake 4 — Long-Running Transactions

Do not perform unnecessary waiting or external operations while holding a transaction open.

Mistake 5 — Swallowing Errors

Bad:

SQL

```
BEGIN CATCH
    PRINT 'Failed';
END CATCH;
```

Better:

EXAMPLE / FLOW

rollback appropriately
+
THROW

Mistake 6 — Using RAISERROR for All New Code

Know it for legacy systems.
Prefer:

EXAMPLE / FLOW

THROW

for modern new error-handling code unless you specifically require a RAISERROR-only capability.

Mistake 7 — Using XACT_ABORT ON but No CATCH Design

XACT_ABORT helps transaction behavior but does not replace deliberate application error handling.

Mistake 8 — Assuming Every Error Makes the Transaction Uncommittable

No.
Check:

SQL

```
XACT_STATE()
```

Mistake 9 — Using @@TRANCOUNT to Detect Committability

@@TRANCOUNT only tells you the nesting count.
It does not tell you:

EXAMPLE / FLOW

committable vs uncommittable

Use:

EXAMPLE / FLOW

XACT_STATE()

for that distinction.

Mistake 10 — Leaving Transactions Open During User Interaction

Never do:

EXAMPLE / FLOW

```
BEGIN TRANSACTION

database update

ask user "Are you sure?"

wait

COMMIT
```

The confirmation should happen before starting the transaction.

Mistake 11 — Assuming Blocking Is Always a Bug

Some blocking is normal.
It becomes a concern when it causes excessive waits or throughput problems.

Mistake 12 — Using NOLOCK to "Fix" Every Blocking Problem

READ UNCOMMITTED/NOLOCK permits dirty reads.
Fix the actual transaction/query/indexing problem rather than sacrificing correctness automatically.

Mistake 13 — Using SERIALIZABLE Everywhere

It provides strong guarantees but can heavily reduce concurrency.
Use it only when the business rule genuinely requires those guarantees.

Mistake 14 — Performing Huge Batch Updates in One Transaction Without Need

A transaction touching millions of rows can generate:

EXAMPLE / FLOW

```
long blocking

large log usage

long rollback time
```

Consider controlled batching if atomicity across the entire set is not required.

Mistake 15 — Catching the Error but Committing Anyway

Do not blindly:

```sql
BEGIN CATCH
    COMMIT;
END CATCH;
```

First understand:

```sql
XACT_STATE()
```

and the business semantics.



Transaction Design Best Practices

Use these rules:

EXAMPLE / FLOW

Define the business unit of work first

Keep transactions short

Use explicit transactions for related multi-statement operations

Use TRY...CATCH

Use SET XACT_ABORT ON where appropriate

Use THROW for modern error propagation

Always design both COMMIT and ROLLBACK paths

Use XACT_STATE when transaction state matters

Know who owns a transaction in nested procedures

Use savepoints for true partial rollback scenarios

Use constraints for data integrity

Use appropriate indexes to reduce lock duration

Access shared resources in consistent order

Do not perform external calls inside transactions unless unavoidable

Do not automatically use the highest isolation level

EXAMPLE / FLOW • CONT.

Recommended Basic Stored Procedure Template

For a procedure that owns its transaction:

SQL

```
CREATE OR ALTER PROCEDURE schema.usp_Operation
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY

        BEGIN TRANSACTION;

        -- Database changes

        COMMIT TRANSACTION;

    END TRY
    BEGIN CATCH

        IF XACT_STATE() <> 0
        BEGIN
            ROLLBACK TRANSACTION;
        END;

        THROW;

    END CATCH;
END;
GO
```

This is the main beginner pattern to remember.

Practical Fundraising Problem — Split Gift

Requirement:
A donor gives:

EXAMPLE / FLOW

$1,000

Split:

EXAMPLE / FLOW

```
$700 → Cancer Research Fund

$300 → Patient Assistance Fund
```

We need:

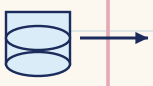
EXAMPLE / FLOW

Gift.TotalAmount = $1,000

and:

EXAMPLE / FLOW

SUM(GiftAllocation)
= $1,000



Transaction Flow

EXAMPLE / FLOW

```
BEGIN TRANSACTION
        |
        ▼
Insert Gift $1,000
        |
        ▼
Capture GiftId
        |
        ├──── Insert $700 allocation
        |
        └──── Insert $300 allocation
        |
        ▼
Validate total allocations
        |
        ├── $1,000 → COMMIT
        |
        └── not $1,000 → THROW → ROLLBACK
```



Example

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_CreateSplitGift
    @TenantId BIGINT,
    @ConstituentId BIGINT,
    @GiftDate DATE,
    @Fund1Id BIGINT,
    @Fund1Amount DECIMAL(18,2),
    @Fund2Id BIGINT,
    @Fund2Amount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @TotalAmount DECIMAL(18,2) =
        @Fund1Amount + @Fund2Amount;
```

SQL • CONT.

```
    IF @Fund1Amount <= 0
       OR @Fund2Amount <= 0
    BEGIN
        THROW 50010,
              'Allocation amounts must be positive.',
              1;
    END;

    BEGIN TRY

        BEGIN TRANSACTION;

        INSERT INTO fundraising.Gift
        (
            TenantId,
            ConstituentId,
            GiftDate,
            TotalAmount
        )
        VALUES
        (
            @TenantId,
            @ConstituentId,
            @GiftDate,
            @TotalAmount
        );

        DECLARE @GiftId BIGINT =
            CONVERT
            (
                BIGINT,
                SCOPE_IDENTITY()
            );

        INSERT INTO fundraising.GiftAllocation
        (
            TenantId,
            GiftId,
            FundId,
            AllocationAmount
        )
        VALUES
        (
            @TenantId,
            @GiftId,
            @Fund1Id,
            @Fund1Amount
        ),
        (
            @TenantId,
            @GiftId,
            @Fund2Id,
            @Fund2Amount
        );

        IF
        (
            SELECT SUM(AllocationAmount)
            FROM fundraising.GiftAllocation
            WHERE GiftId = @GiftId
        ) <> @TotalAmount
        BEGIN
            THROW 50011,
```

SQL • CONT.

```
                    'Gift allocations do not equal the Gift total.',
                    1;
        END;

        COMMIT TRANSACTION;

        SELECT
            @GiftId AS GiftId,
            @TotalAmount AS TotalAmount;

    END TRY
    BEGIN CATCH

        IF XACT_STATE() <> 0
        BEGIN
            ROLLBACK TRANSACTION;
        END;

        THROW;

    END CATCH;
END;
GO
```



Why This Is a Good Transaction Example

It protects one complete business event:

EXAMPLE / FLOW

Gift
+
GiftAllocation rows

If any Fund reference fails:

EXAMPLE / FLOW

foreign key error

or allocation validation fails:

EXAMPLE / FLOW

THROW

the transaction rolls back.



25 Interview Questions and Practical Problems

1. What is a transaction?

A transaction is one logical unit of work.
Its changes are either:

EXAMPLE / FLOW

committed

or:

EXAMPLE / FLOW

rolled back.

Example:

EXAMPLE / FLOW

Create Gift
+
Create GiftAllocation

should often be one transaction.

2. Why do we need transactions?

Transactions prevent partially completed business operations.
If Gift insertion succeeds but allocation fails, a transaction lets SQL Server undo the Gift insertion too.

3. What does ACID stand for?

EXAMPLE / FLOW

Atomicity
Consistency
Isolation
Durability

Atomicity means all-or-nothing.
Consistency means valid state to valid state.
Isolation controls concurrent transactions.
Durability means successfully committed work persists.

4. What is autocommit mode?

Autocommit is SQL Server's normal mode where every individual SQL statement is treated as its own transaction.
If the statement succeeds it commits; if the statement fails it rolls back.

5. What is an explicit transaction?

An explicit transaction is manually started:

```sql
BEGIN TRANSACTION;
```

and manually completed with:

```sql
COMMIT;
```

or:

```sql
ROLLBACK;
```

6. What is an implicit transaction?

With:

```sql
SET IMPLICIT_TRANSACTIONS ON;
```

SQL Server automatically begins transactions for qualifying statements, but the developer must explicitly commit or roll them back.

7. What does BEGIN TRANSACTION do?

It marks the start of an explicit transaction and increments:

EXAMPLE / FLOW

@@TRANCOUNT

by one.

8. What does COMMIT do?

It marks successful transaction completion.
If it is the outermost transaction, changes become permanent and transaction resources are released.
An inner commit only decreases @@TRANCOUNT.

9. What does ROLLBACK do?

A full:

```sql
ROLLBACK TRANSACTION;
```

undoes the active transaction's work.
A rollback to a savepoint only undoes work after that savepoint.



10. What is @@TRANCOUNT?

@@TRANCOUNT returns the number of transaction BEGIN levels active on the connection.

EXAMPLE / FLOW

```
BEGIN → +1
COMMIT → -1
full ROLLBACK → 0
savepoint rollback → unchanged
```



11. Does SQL Server support true nested transactions?

Not independently manageable ones.
Inner BEGIN statements increase @@TRANCOUNT, but inner COMMIT does not independently make changes permanent.
Only the outermost commit completes the transaction.



12. What is SAVE TRANSACTION?

SAVE TRANSACTION creates a savepoint inside an active transaction.
Example:

```sql
SAVE TRANSACTION BeforeAllocation;
```

Then:

```sql
ROLLBACK TRANSACTION BeforeAllocation;
```

rolls back only work after that savepoint when transaction state permits it.



13. What is TRY...CATCH?

It is SQL Server's structured run-time error-handling mechanism.
Statements run inside:

EXAMPLE / FLOW

TRY

and caught errors transfer execution to:

EXAMPLE / FLOW

CATCH.

14. Which ERROR functions are commonly used?

EXAMPLE / FLOW

ERROR_NUMBER()

ERROR_MESSAGE()

ERROR_SEVERITY()

ERROR_STATE()

ERROR_LINE()

ERROR_PROCEDURE()

They describe the caught error.

15. What is THROW?

THROW raises an error.
Inside CATCH:

```sql
THROW;
```

rethrows the original caught error.
For modern application development, it is generally preferred over RAISERROR.

16. What is the difference between THROW and RAISERROR?

RAISERROR is the older mechanism and has features such as explicit severity and formatted messages.
THROW is the modern default, supports straightforward rethrowing, and honors SET XACT_ABORT.
RAISERROR does not honor SET XACT_ABORT.

17. What is SET XACT_ABORT ON?

It tells SQL Server that covered run-time errors should terminate and roll back the active transaction rather than simply leaving execution to continue after a failed statement.
It is commonly used in transaction-oriented stored procedures.

18. What does XACT_STATE() return?

EXAMPLE / FLOW

```
0
→ No active transaction

1
→ Active and committable

-1
→ Active but uncommittable
```

19. What is the difference between XACT_STATE and @@TRANCOUNT?

@@TRANCOUNT tells you:

EXAMPLE / FLOW

transaction nesting count.

XACT_STATE() tells you:

EXAMPLE / FLOW

whether the transaction can still commit.

20. What is an uncommittable transaction?

It is an active transaction with:

EXAMPLE / FLOW

XACT_STATE() = -1

SQL Server will not allow it to commit.
It needs to be rolled back.

21. What is blocking?

Blocking occurs when one session must wait for another transaction to release a conflicting lock.

Blocking is a normal part of concurrency, but long transactions can make it harmful.

22. What is a deadlock?

A deadlock is a circular wait.
Example:

EXAMPLE / FLOW

Transaction A owns Gift and waits for Fund.

Transaction B owns Fund and waits for Gift.

SQL Server detects the cycle and rolls back one transaction as the deadlock victim so the other can continue.

23. How do you reduce deadlocks?

Common practices:

EXAMPLE / FLOW

Keep transactions short

Access tables/resources consistently

Use good indexes

Avoid unnecessary high isolation

Avoid user interaction during transactions

Minimize network round-trips

24. Query Problem — Create Gift and Allocation Atomically

SQL

```
SET XACT_ABORT ON;

BEGIN TRY

    BEGIN TRANSACTION;

    INSERT INTO fundraising.Gift
    (
        TenantId,
        ConstituentId,
        GiftDate,
        TotalAmount
    )
    VALUES
```

SQL • CONT.

```
    (
        1,
        1001,
        '2026-09-21',
        1000
    );

    DECLARE @GiftId BIGINT =
        CONVERT
        (
            BIGINT,
            SCOPE_IDENTITY()
        );

    INSERT INTO fundraising.GiftAllocation
    (
        TenantId,
        GiftId,
        FundId,
        AllocationAmount
    )
    VALUES
    (
        1,
        @GiftId,
        300,
        1000
    );

    COMMIT TRANSACTION;

END TRY
BEGIN CATCH

    IF XACT_STATE() <> 0
    BEGIN
        ROLLBACK TRANSACTION;
    END;

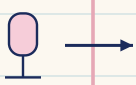
    THROW;

END CATCH;
```

If either insert fails:

EXAMPLE / FLOW

both are rolled back.



25. Full Interview Problem

Requirement

A stored procedure may run by itself or be called inside another transaction.
How should it behave?
The main reasoning is:

EXAMPLE / FLOW

1. Check @@TRANCOUNT.

2. If no transaction exists:
   begin one and remember
   that this procedure owns it.

3. If a transaction already exists:
   do not pretend you own
   an independent nested transaction.

4. Optionally create a savepoint.

5. On success:
   commit only if you started the transaction.

6. On failure:
   inspect XACT_STATE().

7. Roll back your own transaction,
   or use the savepoint when safe.

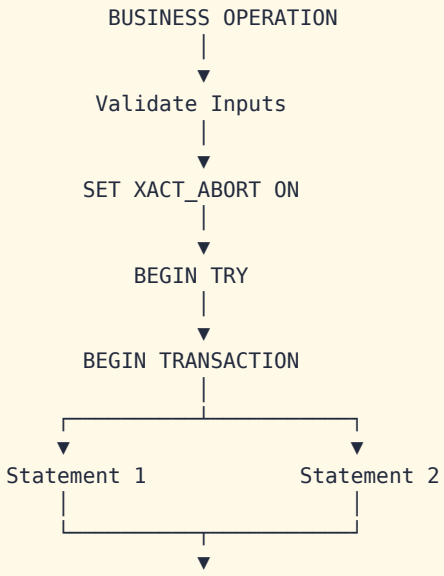
8. THROW the error to the caller.

This demonstrates the key SQL Server rule:

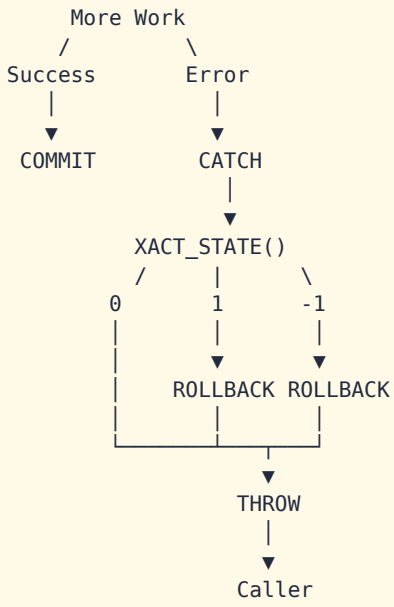
Nested BEGIN TRANSACTION statements do not create independently managed transactions.

Final Master Diagram

EXAMPLE / FLOW



EXAMPLE / FLOW • CONT.



Final Easy Memory Rules

EXAMPLE / FLOW

Transaction
→ One logical unit of work

EXAMPLE / FLOW

BEGIN TRANSACTION
→ Start explicit transaction

EXAMPLE / FLOW

COMMIT
→ Keep changes

EXAMPLE / FLOW

ROLLBACK
→ Undo changes

EXAMPLE / FLOW

SAVE TRANSACTION
→ Partial rollback point

EXAMPLE / FLOW

```
@@TRANCOUNT
→ How many transaction BEGIN levels?
```

EXAMPLE / FLOW

```
XACT_STATE()
→ Is the transaction usable?
```

EXAMPLE / FLOW

```
TRY...CATCH
→ Handle run-time errors
```

EXAMPLE / FLOW

```
THROW
→ Raise or rethrow error
```

EXAMPLE / FLOW

```
SET XACT_ABORT ON
→ Safer atomic behavior for many run-time errors
```

EXAMPLE / FLOW

```
Atomicity
→ All or nothing
```

EXAMPLE / FLOW

```
Consistency
→ Valid state to valid state
```

EXAMPLE / FLOW

```
Isolation
→ Control concurrent visibility/interference
```

EXAMPLE / FLOW

```
Durability
→ Committed work persists
```

EXAMPLE / FLOW

```
Blocking
→ Transaction waits for another
```

EXAMPLE / FLOW

```
Deadlock
→ Transactions wait on each other in a cycle
```

EXAMPLE / FLOW

```
Best transaction
→ Correct + short + predictable
```

The most important production pattern to remember is:

SQL

```sql
SET XACT_ABORT ON;

BEGIN TRY

    BEGIN TRANSACTION;

    -- Keep only the database work
    -- that must succeed together here.

    COMMIT TRANSACTION;

END TRY
BEGIN CATCH

    IF XACT_STATE() <> 0
    BEGIN
        ROLLBACK TRANSACTION;
    END;

    THROW;

END CATCH;
```

And the most important design question is:



What is the smallest complete business operation that must either succeed entirely or leave no partial changes behind?

For your FundraisingDb, Gift + GiftAllocation is the clearest example: they describe one donation event, so transaction design should prevent the database from permanently keeping only half of that operation.